

Build and Run Instructions

1. Open 3 cmd prompts.
2. Always run the server first. On the first cmd, type the command "python -m server.server --verbose" to open the server.
3. This step is for running the first player (client 1). On the other cmd, type the command "python -m client.ui --verbose" to open the client.
4. Repeat step 3 for the second player (client 2).

Work Distribution

The following table outlines the responsibilities and features assigned to each team member for this project.

Task / Feature	Member 1 (Greg Avila)	Member 2 (Joshua Del Mundo)	Member 3 (Ulrich Gonzales)	Member 4 (Mark Sandoval)
TCP Server: connection handling, framing, dispatch	X			
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic			X	

Turn & phase engine (all phases/steps, transitions)				X
Priority & Stack logic, spell/ability resolution	X			
Combat system (attackers, blockers, damage)	X			
Client implementation & state rendering			X	
PDU serialisation/deserialisation (all 25 PDU types)				X
Error handling, PING/PONG heartbeat, disconnect logic	X			
Verbose mode (client + server PDU logging, toggle on/off)			X	
Testing & interoperability		X		
README / documentation / AI disclosure		X		

AI Declaration

Used ChatGPT to learn about MTG, validation of implementation, and testing of code.

Limitation and Deviations from the RFC

1. Mana Payment Bypass

- The server contains a flag named `DEBUG_FORCE_LIGHTNING_BOLT_MANA` which is actively set to `True`. This completely bypasses the mana payment check for the Lightning Bolt card, allowing it to be cast without sufficient resources.

2. Forced Opening Hands

- The `DEBUG_SEED_CARD_ID` flag is active and set to `lightning_bolt_003` for `player_1`. This circumvents the RFC's requirement for randomized deck shuffling by forcing that specific card instance into the player's opening hand during setup.

3. Triggered Abilities

- The RFC mandates a system for trigger detection, requiring the server to ask players for `TRIGGER_ORDER` when simultaneous triggers fire, and `TRIGGER_CHOICE` for optional "you may" triggers. While the client UI is programmed to send these responses, the server's `_dispatch_action` method completely lacks the logic to receive or process them.

4. Activated Abilities

- The RFC notes that players may activate non-mana abilities whenever they hold priority. The client wrapper includes an `ActivateAbility` PDU, but the server does not handle this action and will reject it as unimplemented.

5. First Strike Damage

- The RFC defines an optional `FIRST_STRIKE_DAMAGE` phase where only creatures with first strike or double strike deal damage before standard combat. The server attempts to route to this phase by calling `self._combat_damage(first_strike=True)`. However, the `_combat_damage` function itself ignores the `first_strike` argument and applies damage for all attacking creatures without filtering for the specific ability.

6. Timeout Enforcement

- The RFC dictates that the server must enforce the `time_limit_ms` advertised in priority grants, broadcasting a `GAME_OVER` due to a `DISCONNECT` if a player fails to respond. While the server echoes heartbeat Ping messages, it lacks any asynchronous timer loops or background threads to actually enforce these priority deadlines.